



STAGE SGSI

-

CRÉATION D'UN CATALOGUE DES APIs

Cahier des charges

Auteur :

KHEIRALLAH Cédric

Superviseur :

MINNE Frédéric

7 juillet 2026

1 Introduction

1.1 Objectifs et Public Cible

Nous souhaitons créer un catalogue des APIs^{1.4} disponibles pour les consultants externes au SGSI ainsi que pour les développeurs internes permettant de connaître leurs intérêts pour certaines APIs.

1.2 Fonctionnalités attendues

Le produit final idéal doit présenter les capacités suivantes :

- Présenter les APIs visuellement de manière similaire à l'interface d'API Manager^{1.4} (nom, icône potentielle, version, temps de la dernière mise à jour, tags potentiels) et possédant une barre de recherche ainsi que des filtres de catégorie pour faciliter la navigation du marketplace.
- Chaque API doit rediriger vers une page présentant les informations plus détaillées de l'API (swagger téléchargeable, numéro de révision, documentation, exemples de réponses, interface offrant la possibilité de demander l'accès à l'API).
- Être capable de se mettre à jour automatiquement de manière régulière ainsi que de pouvoir être mis à jour manuellement par ses administrateurs en cas de besoin.
- Offrir la possibilité de cacher certaines APIs pour certains utilisateurs selon leurs droits d'accès.
- Offrir la possibilité aux administrateurs de blacklister/exclure certaines APIs.
- Permettre la suppression manuelle de certaines APIs.

1.3 Contraintes

- Le catalogue doit rester en lecture seule pour les utilisateurs finaux qui ne peuvent pas tester les APIs.
- L'application doit être documentée tant au niveau de son architecture et organisation qu'au niveau du code et de son utilisation.
- Ce projet se fera de manière itérative semaines par semaine tout au long du stage.

1.4 Définitions, Acronymes et Abréviations

- **API** : Application Programming Interface
- **API Manager** : Outil permettant de gérer les APIs
- **API Controller** : Outil en ligne de commande permettant de communiquer avec API Manager

2 User Stories

2.1 Besoins des Utilisateurs

2.2 Besoins des Administrateurs

3 Fonctionnalités et Exigences du Système

3.1 Exigences Fonctionnelles

3.2 Exigences Techniques

3.3 Autres Exigences

4 Processus de Développement

5 Appendices

5.1 Technologies et Frameworks

- **Django** : le framework web sera probablement Django car il est déjà connu et utilisé dans le service et permet de lier facilement les URLs aux différentes vues qui seront générées automatiquement en utilisant la fonctionnalité de template de Django.
- **Cron** : le planificateur automatique utilisé sera cron car il est déjà connu et fiable. Il permettra de régénérer automatiquement le catalogue des APIs à intervalle régulier.
- **Python** : le langage permettra de créer facilement les programmes d'importation et d'extraction d'autant plus que le framework Django est déjà lui-même basé sur ce langage, cela facilitera la maintenance de l'application.
- **Docker** : nous isolerons dans des conteneurs docker les composants suivants :
 - le framework Django
 - le planificateur automatique cron
 - l'application d'importation et d'extraction